

Kolokwium 2 - Java/C#

Imię:

Nazwisko:

Na napisanie testu jest 85 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem).

1. Jaki będzie wyniki kompilacji i wykonania klasy Enumy:

```
public enum Enumy {  
    ONE {  
        public Enumy theOther() {  
            return TWO;  
        }  
    },  
    TWO {  
        public Enumy theOther() {  
            return ONE;  
        }  
    };  
    public static void main(String[] args) {  
        for (Enumy e : Enumy.values()) {  
            System.out.println(e.theOther());  
        }  
    }  
}
```

- plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- klasa się uruchomi i wypisze:

2. Zaznacz wszystkie sytuacje w których wykonanie wątku testThread będzie natychmiast przerwane kiedy inny wątek wywoła testThread.interrupt()

- TimeUnit.MILLISECONDS.sleep(300);
- System.in.read();
- wait();
- oczekiwanie na wejście do bloku synchronized.

3. Dla poniższego main'a zaznacz wszystkie poprawne odpowiedzi:

```
public static void main(String[] args) {  
    try {
```

```

        new Thread() {
            @Override
            public void run() {
                throw new RuntimeException();
            }
        }.start();
    } catch (RuntimeException e) {
        System.out.println("Exc");
    }
    System.out.println("The End");
}

```

- a. kod nie skompiluje się,
- b. wykonanie wątku zostanie przerwane przez wyjątek,
- c. program wypisze Exc,
- d. program wypisze The End.

4. Jaki będzie wyniki kompilacji i wykonania poniższego main'a (w klasie z odpowiednimi importami):

```

public static void main(String[] args){
    System.out.println(
        CharBuffer.allocate(12).put("Hello world!").toString());
}

```

- a. plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- b. zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- c. klasa się uruchomi i wypisze:

5. Zaznacz wszystkie poprawne stwierdzenia:

- a. Wszystkie obiekty klas implementujących Serializable dają się serializować.
- b. Jeśli klasa posiada pole nieserializowalnego typu, to obiektów tej klasy nie da się serializować.
- c. Interfejs Serializable definiuje metody void writeObject(ObjectOutputStream out) i void readObject(ObjectInputStream in)
- d. Metody readObject i writeObject (jak wyżej) nie będą użyte podczas serializacji jeśli nie będą prywatne.

6. Podaj cztery adnotacje używane w Java:

7. Zaznacz wszystkie poprawne stwierdzenia:

- a. assert to słowo kluczowe języka Java pozwalające na testowanie prawdziwości warunków w czasie wykonania programu.
- b. assert false; powoduje wyrzucenie wyjątku typu RuntimeException
- c. Asercje są automatycznie włączone w trakcie wykonania bytecode'u przez JVM.
- d. Jeśli przetwarzanie asercji jest wyłączone to kod: assert a++ == 0; spowoduje inkrementację zmiennej a i nigdy nie zgłosi błędu.

8. Dla poniższego kodu zaznacz wszystkie prawidłowe stwierdzenia:

```
import java.util.EnumSet;
public class Test1 {
    enum Day { SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY }
    public static void main(String[] args) {
        for (Day d : EnumSet.range(Day.SUNDAY, Day.MONDAY))
            System.out.println(d);
    }
}
```

- a. Kod nie skompiluje się, bo Day nie jest kolekcją
- b. Kod wykona się i wypisze "SUNDAY\nMONDAY\n"
- c. Kod wykona się i rzuci wyjątek java.lang.IllegalArgumentException
- d. Kod nie skompiluje się bo Day nie jest statyczne

9. Dla poniższego kodu zaznacz wszystkie prawidłowe stwierdzenia:

```
import java.io.*;
public class Test3 {
    static public final class Enum implements java.io.Serializable
    {
        public static final Enum TRUE = new Enum ();
        public static final Enum FALSE = new Enum ();
        private Enum () {}
    }
    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        ByteArrayOutputStream bout = new ByteArrayOutputStream ();
        ObjectOutputStream out = new ObjectOutputStream (bout);
        Enum e1 = Enum.TRUE;
        out.writeObject (e1);
        out.flush ();
        ByteArrayInputStream bin =
new ByteArrayInputStream (bout.toByteArray ());
        ObjectInputStream in = new ObjectInputStream (bin);
        Enum e2 = (Enum) in.readObject ();
        System.out.println ((e2 == Enum.TRUE || e2 == Enum.FALSE));
    }
}
```

}

- a. Kod skompiluje się i wypisze "true"
- b. Kod skompiluje się i wypisze "false"
- c. Kod nie wykona się i rzuci wyjątek
- d. Kod nie skompiluje się bo Enum nie może być serializowalna

10. Wskaż wszystkie poprawne stwierdzenia:

- a. BlockingQueue to struktura danych implementująca synchronizowaną kolejkę FIFO
- b. Wątek który czeka na obiekcie (za pomocą metody wait()) może być wznowiony poprzez wywołanie na wątku metody restart()
- c. Wątek który czeka na obiekcie (za pomocą metody wait()) może być wznowiony poprzez wywołanie na wątku metody notify()
- d. Wątek który czeka na obiekcie (za pomocą metody wait()) może być wznowiony poprzez wywołanie na tym obiekcie metody notify()

11. Wskaż prawidłowe zdania dotyczące Bytecode:

- a. Bytecode to forma instrukcji które wykonuje JVM
- b. Bytecode jest językiem skryptowym
- c. Każdy bytecode ma wielkość 2 bajtów
- d. Bytecode dla JVM jest tym czym czym instrukcje assemblerowe dla komputera

12. Napisz co wypisuje następujący bytecode:

```
public class Test5 extends java.lang.Object{
public Test5();
Code:
    0:  aload_0
    1:  invokespecial    #8; //Method java/lang/Object."<init>":()V
    4:  return

public static void main(java.lang.String[]);
Code:
    0:  iconst_1
    1:  istore_1
    2:  goto    14
    5:  iload_1
    6:  iload_1
    7:  iconst_1
    8:  iadd
    9:  imul
   10:  istore_1
```

```

11:  iinc      1, 1
14:  iload_1
15:  iconst_5
16:  if_icmplt      5
19:  getstatic      #16; //Field java/lang/System.out:Ljava/io/PrintStream;
22:  iload_1
23:  invokevirtual  #22; //Method java/io/PrintStream.println:(I)V
26:  return
}

```

13. Konstruktor Thread(Runnable x, String s) tworzy nowy wątek (gotowy do uruchomienia) i ustawia jego nazwę osiągalną przez metodę getName(). Podaj wynik kompilacji i uruchomienia następującego programu:

```

class A implements Runnable {
    public void run() {System.out.print(Thread.currentThread().getName());}
}
class B implements Runnable {
    public void run() {
        new A().run();
        new Thread(new A(),"2").run();
        new Thread(new A(),"3").start();
    }
}
class C {
    public static void main (String[] args) {
        new Thread(new B(),"1").start();
    }
}

```

- a. Wypisuje: 111
- b. Wypisuje: 112
- c. Wypisuje: 122
- d. Wypisuje: 123
- e. Wypisuje: 113
- f. Wypisuje: 133
- g. Nie da się jednoznacznie określić.
- h. Rzuca wyjątek w trakcie uruchomienia.

14. W jakim celu zaimplementowane są metody invokeAndWait, invokeLater itp.

15. Które zdania są prawdziwe:

```
import java.io.*;
public class Test4 {
    public static void main(String[] args) {
        BufferedReader in = new BufferedReader(new
            InputStreamReader(System.in));
        System.out.println("Enter text : ");
        String str = in.readLine();
        System.out.println("You entered a String : " + str);
    }
}
```

- a. Jeśli ktoś wpisze "a\n" program wypisze "You entered a String : a\n"
- b. Program rzuci wyjątek (wskaż gdzie)
- c. Program nie skompiluje się (wskaż linię)
- d. Program będzie oczekiwał na interakcję użytkownika

16. Wskaż prawdziwe zdania dotyczące kodu w C#:

```
using System;
class Program
{
    public class Sth
    {
        int a;
        public Sth(int a)
        {
            this.a = a;
        }
        static public Sth operator ++(Sth a)
        {
            return new Sth(++a.a);
        }
        static public implicit operator string(Sth a) {
            return "" + a;
        }
    }
    static void Main(string[] args)
    {
        Sth a = new Sth(1);
        System.Console.WriteLine(a);
    }
}
```

- a. Program się nie skompiluje ponieważ nie jest zdefiniowany operator --
- b. Program się nie skompiluje ponieważ dla klasy Sth nie zdefiniowano metody ToString()
- c. Program się skompiluje i wypisze "1"
- d. Program się skompiluje i wyrzuci StackOverflowException w operatorze rzutowania na string

17. Wskaż prawdziwe zdania dotyczące C#:

- a. Słowo kluczowe explicit sprawia, że rzutowanie musi odbywać się jawnie

- b. Słowo kluczowe `implicit` sprawia, że rzutowanie nie musi odbywać się jawnie
- c. Nie można definiować operatorów rzutowania z typów podstawowych na typy zdefiniowane przez użytkownika
- d. Nie można definiować operatorów rzutowania `implicit` z typów zdefiniowanych przez użytkownika na typy podstawowe

18. Wskaż prawdziwe zdania dotyczące kodu w C#:

```
using System;
class Program
{
    sealed public class Sth
    {
        int a;
        public Sth(int a)
        {
            this.a = a;
        }
    }
    sealed public class SthBetter : Sth
    {
        int b;
        public SthBetter(int b)
        {
            this.b = b;
        }
    }
    static void Main(string[] args)
    {
        Sth a = new Sth(1);
        SthBetter b = new SthBetter(1);
        System.Console.WriteLine(a.a);
        System.Console.WriteLine(b.b);
    }
}
```

- a. Kod nie skompiluje się ponieważ klasa `Sth` nie posiada konstruktora domyślnego
- b. Kod skompiluje i wypisze `"1\n1\n"`
- c. Kod nie skompiluje się ponieważ nie można dziedziczyć po klasie która jest `sealed`
- d. Kod nie skompiluje się ponieważ klasa która jest `sealed` nie może po niczym dziedziczyć

19. Wskaż prawdziwe zdania dotyczące kodu w C#:

```
using System;
class Program
{
    public class BaseClass
    {
        public int a;
        public int b;
    }
}
```

```

public interface Iface1
{
    void WriteOne();
}
public interface Iface2
{
    void WriteTwo();
}
public class ClassA : Iface1, Iface2, BaseClass
{
    public ClassA(int a, int b)
    {
        this.a = a;
        this.b = b;
    }
    public void WriteOne() { System.Console.WriteLine(a); }
    public void WriteTwo() { System.Console.WriteLine(b); }
}
static void Main(string[] args)
{
    ClassA x = new ClassA(1, 1);
    x.WriteOne();
    x.WriteTwo();
}
}

```

- a. Program nie skompiluje się bo interfejs może składać się jedynie z sygnatur metod delegacji i zdarzeń (event)
- b. Program nie skompiluje się bo klasa nie może dziedziczyć po więcej niż jednym interfejsie
- c. Program skompiluje się i wypisze "1\n1\n"
- d. Program nie skompiluje się ponieważ w liście klas po których dziedziczy ClassA klasy bazowe muszą wystąpić przed wszystkimi interfejsami

20. Wskaż prawdziwe zdania dotyczące C#:

- a. Akcesor "get" może być użyty jedynie do zwrócenia elementu
- b. Akcesor "set" może być użyty jedynie do przypisania wartości do elementu
- c. Właściwości można przekazać funkcji jako referencje
- d. Akcesor "get" musi kończyć wykonanie albo poprzez return albo poprzez throw

21. Wskaż prawdziwe zdania dotyczące C#:

- a. Słowo kluczowe this jest wykorzystywane przy definiowaniu indeksera
- b. Słowo kluczowe value jest wykorzystywane przy definiowaniu wartości przypisywanej przez indeks "set"
- c. Indeksery nie mogą być przeciążane
- d. Indeksery nie muszą być indeksowane poprzez "inty"

22. Wskaż prawdziwe zdania dotyczące C#:

- a. Słowa kluczowe struct, class, interface, enum, służą definiowaniu typów użytkownika
- b. Typ zdefiniowany jako class, delegate, array albo interface jest "typem referencyjnym", tzn do czasu przypisania obiektowi tego typu pewnej instancji tegoż typu, obiekt ten ma przypisany null
- c. Przy definiowaniu zmiennych kompilator sam dedukuje typ zmiennej
- d. Zmienna zdefiniowana w programie może zmieniać raz nabyty typ na inny dowolną ilość razy w programie

23. Wskaż prawdziwe zdania dotyczące kodu w C#:

```
class SampleCollection<T>
{
    private T[] arr = new T[100];
    public T this[int i, int j]
    {
        get
        {
            return arr[i + j];
        }
        set
        {
            arr[i + j] = value;
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        SampleCollection<string> sc = new SampleCollection<string>();
        sc[0,1] = "Hello, World";
        System.Console.WriteLine(sc[1,0]);
    }
}
```

- a. Program skompiluje się i wypisze "Hello, World!"
- b. Program skompiluje się, ale wyrzuci NullPointerException
- c. Program nie skompiluje się ponieważ indexer nie może przyjmować wielu argumentów
- d. Program skompiluje się i wypisze "null"

24. Jaki będzie (najprawdopodobniej) wyniki kompilacji i wykonania klasy Test:

```
class Test
{
    static Test(){
        Console.WriteLine("*");
    }
    internal Test(){
        Console.WriteLine("+");
    }
}
```

```

    }
    ~Test(){
        Console.WriteLine("-");
    }
    public static void Main (string[] args)
    {
        {
            Test t;
        }
        {
            Test t = new Test();
        }
        {
            GC.SuppressFinalize(new Test());
        }
    }
}

```

- a. plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- b. zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- c. klasa się uruchomi i wypisze:

25. Klasa B dziedziczy po klasie A (która z kolei może dziedziczyć po innych klasach), i obie klasy implementują metodę void Test() - zaznacz które pary deklaracji powodują ostrzeżenie, które błąd czasu kompilacji, a które skompilują się bez problemów (zakładając że dziedziczenie wśród przodków A jest ok).

- a. new void Test() w A i override void Test() w B
- b. new void Test() w A i virtual void Test() w B
- c. override void Test() w A i override void Test() w B
- d. new virtual void Test() w A i new virtual void Test() w B
- e. new virtual void Test() w A i override void Test() w B

26. Jaki będzie wynik kompilacji i wykonania klasy StructTest:

```

using System;
public interface ITest {
    int X {get; set;}
}
public struct StructTest : ITest
{
    public int X { get; set; }
    public static void Test1 (StructTest t)
    {
        t.X++;
    }
}

```

```

        Test2(t);
    }
    public static void Test2 (ITest i)
    {
        i.X++;
        Test3((StructTest)i);
    }
    public static void Test3 (StructTest t)
    {
        t.X++;
    }
    public static void Main(string[] args)
    {
        StructTest t = new StructTest();
        Test1(t);
        Console.WriteLine(t.X);
        Test2(t);
        Console.WriteLine(t.X);
        Test3(t);
        Console.WriteLine(t.X);
    }
}

```

- plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- klasa się uruchomi i wypisze:

27. Jaki będzie wynik kompilacji i wykonania klasy Delegates

```

using System;
public class Delegates
{
    delegate string del(string s);
    public static void Main (string[] args)
    {
        del a = null;
        a += s => s + s;
        Console.WriteLine(a("kot"));
        a += a;
        Console.WriteLine(a("kot"));
        a += s => {Console.WriteLine("++" + s + "++"); return s;};
        Console.WriteLine(a("kot"));
        a += s => {throw new Exception();};
        try {
            Console.WriteLine(a("kot"));
        }catch(Exception e){
            Console.WriteLine("exc");
        }
    }
}

```

```

    }
}

```

- a. plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- b. zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- c. klasa się uruchomi i wypisze:

28. Wypisz wszystkie interfejsy które muszą zostać zaimplementowane w klasie Test, aby poniższa pętla wykonała się poprawnie

```
foreach(string s in new Test())
    Console.WriteLine(s);
```

29. Zaznacz wszystkie poprawne kontynuacje zdania:

Klasa może implementować dwa interfejsy

- a. które deklarują metody o tej samej nazwie i liście parametrów, a różnym typie wyniku.
- b. z których w jednym P to oznaczenie właściwości, a w drugim metody bezargumentowej.
- c. wtedy i tylko wtedy, kiedy każda z metody każdego z interfejsów może byćwołana przez referencję danej klasy.
- d. dziedziczące po wspólnym przodku.

30. Jaki będzie wyniki kompilacji i wykonania klasy RefOut:

```
class RefOut
{
public static void Test1 (ref int i, out int j)
{
Test2 (out i, ref j);
}
public static void Test2 (out int i, ref int j)
{
Console.WriteLine("i = " + i + " j = " + j);
}
public static void Main (string[] args)
{
Test1 (ref 1, out 2);
}
}
```

- a. plik nie skompiluje się (zaznacz w kodzie gdzie jest pierwszy błąd i jaki)
- b. zostanie wyrzucony wyjątek (zaznacz w kodzie skąd zostanie wyrzucony)
- c. klasa się uruchomi i wypisze: